



IE3064 - Embedded Systems Design

3rd Year, 1st Semester

Line Following Arduino Car

**As a partial response to the mandate for
BSc (Hons) in Computer Systems Engineering**

Submitted by

K.M.T.P.Kulasinha – IT23749566

LKCM Perera – IT23619494

Group Number: Group - 05

Date: 2026 / 04 / 27

Table of Contents

1. Project Introduction	3
2. Hardware Design and Component Setup	4
2.1 Hardware Components	4
2.2 Overall System Description	4
2.3 Pin Mapping	5
2.4 Circuit Diagram	6
3. Firmware Design	7
3.1 Program Organization.....	7
3.2 Register Allocation	7
3.3 PWM-Based Motor Speed Control.....	8
4. System Behavior Model.....	9
5. Algorithm Flow Summaries	10
5.1 Line Following Process	10
5.2 Parking Detection Process	11
5.3 Parking Execution Process.....	12
6. System Implementation	13
6.1 Line Following Implementation	13
6.2 Parking Detection Implementation	13
6.3 Parking Maneuver Implementation.....	13
7. Testing, Results, and Evaluation	14
7.1 Test Cases and Results.....	14
7.2 System Performance Review	15
7.3 Issues Faced and Improvements	15
8. Physical Build Evidence	16
8.1 Top View	16
8.2 Wiring Layout.....	16
9. Discussion.....	17
10. Conclusion	17
11. References.....	18
12. Appendix.....	18
Individual Contribution	18

1. Project Introduction

This project was carried out for the IE3064 Embedded Systems Design module. The aim of the assignment was to design and implement a battery-powered autonomous car using an Arduino Uno that could perform two major tasks: follow a line track and park inside the correct parking bay. A major constraint of the assignment was that the firmware had to be implemented entirely in AVR Assembly language.

The developed system uses an Arduino Uno based on the ATmega328P microcontroller, a TB6612FNG motor driver, two TCRT5000 line sensors, an IR flame sensor for parking detection, two DC motors, and a battery-powered chassis. The vehicle was designed to move autonomously along a marked black line, detect the correct parking condition, and then perform a parking maneuver that places the robot within the selected bay.

The final implementation successfully achieved both required operating stages. The robot was able to move forward, follow straight and curved sections of the line, detect marker transitions for entering and leaving the parking stage, stop safely when the line was lost for too long, and park inside the correct bay after detecting the selected signal. Through this project, important embedded systems concepts such as low-level port programming, PWM motor control, interrupt handling, sensor interfacing, marker timing, and structured Assembly programming were learned in a practical way.

2. Hardware Design and Component Setup

2.1 Hardware Components

The final system was built using the following main hardware components.

Component	Description	Purpose
Arduino Uno (ATmega328P)	Microcontroller board	Main controller
TB6612FNG	Dual motor driver	Controls two DC motors
2 × 18650 batteries (3.7 V each)	Battery pack	Main power source
2 × DC motors	Drive motors	Movement of the vehicle
2 × TCRT5000 sensors	Reflective line sensors	Line detection
IR flame sensor	Digital parking sensor	Parking bay detection
Chassis with wheels	Robot frame	Mechanical support
Connecting wires and battery connector	Interconnections	Power and signal routing

2.2 Overall System Description

The system can be understood using three main layers: the input layer, the processing layer, and the output layer.

The input layer consists of the two TCRT5000 sensors and the IR flame sensor. The TCRT5000 sensors are used to detect the black line on the floor, while the IR flame sensor is used as the parking detection input. The processing layer is the Arduino Uno, which executes all logic in AVR Assembly language. It reads sensor values, makes movement decisions, and activates the correct parking routine when required. The output layer consists of the TB6612FNG motor driver and the two DC motors. These provide the actual motion of the robot based on the control signals from the microcontroller. This layered explanation is similar in purpose to the reference report's system overview section, but the hardware and signal mapping here are specific to your system.

The system operates as a closed-loop embedded system. The robot continuously reads the line sensors, compares the sensor states, and adjusts its motor control signals to remain on the track. When the parking condition is detected, the system switches from normal line following to the parking sequence. This makes the vehicle capable of autonomous behavior using low-level hardware control.

2.3 Pin Mapping

The following pin configuration was used in the implementation.

Pin	Function
D10	PWMA
D9	AIN2
D8	AIN1
D7	BIN1
D6	BIN2
D5	PWMB
A0	Left line sensor
A1	Right line sensor
D3	IR interrupt sensor

At low level in AVR Assembly, the sensor and control pins are handled through the ATmega328P registers. The left and right line sensors are treated through PC0 and PC1, the parking interrupt input uses PD3 / INT1, motor direction uses PB0, PB1, PD6, and PD7, and hardware PWM uses PB2 and PD5.

2.4 Circuit Diagram

The complete schematic includes the Arduino Uno, TB6612FNG motor driver, two TCRT5000 line sensors, the IR flame sensor, battery connector, and the two DC motors. The battery positive terminal is connected to the motor driver’s motor supply input, while all grounds are common. The driver receives PWM and direction control signals from the Arduino, and the sensors provide the input signals required for line following and parking detection.

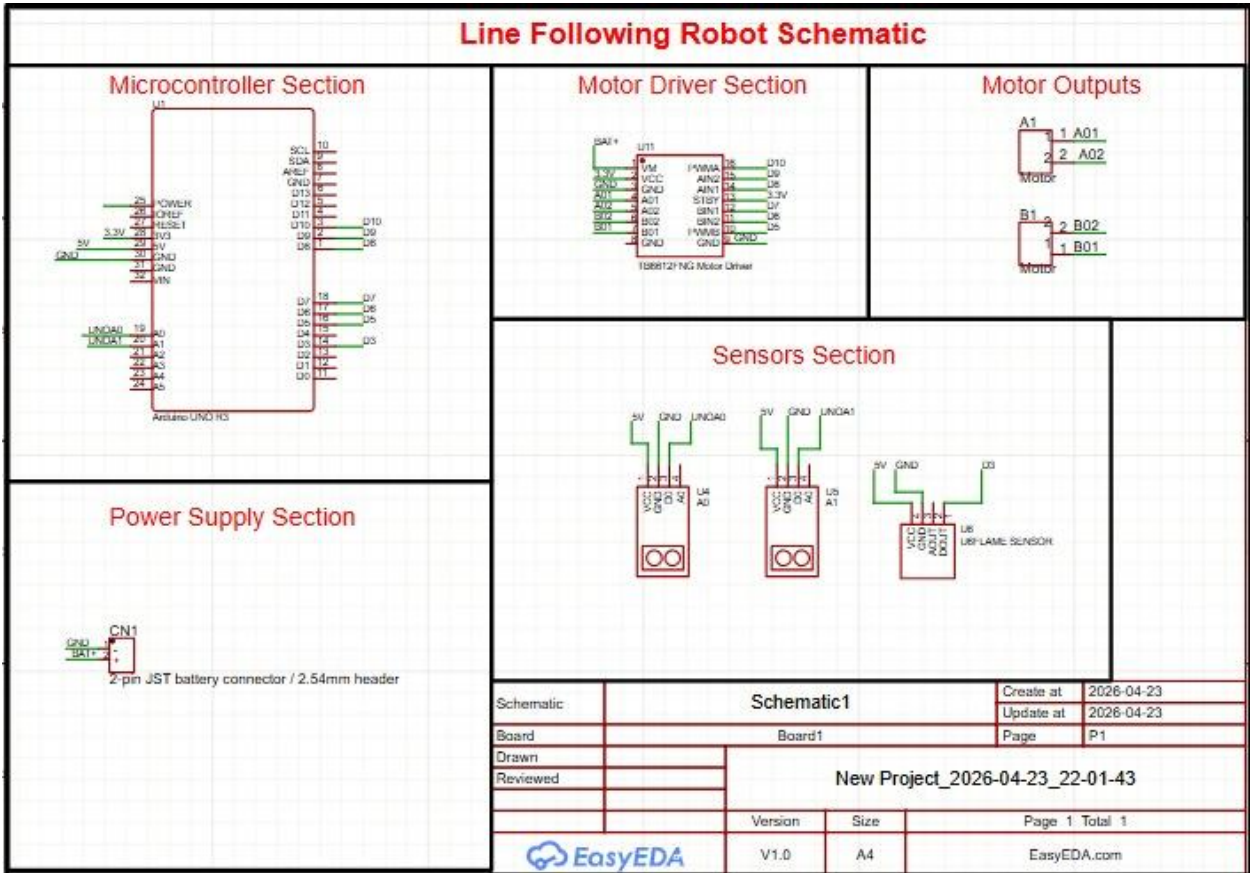


Figure 1: Circuit Schematic of the Line Following and Parking Robot

3. Firmware Design

3.1 Program Organization

The firmware was developed fully in AVR Assembly language. The program begins with a vector table containing the reset vector and the interrupt vector for INT1. After reset, the program initializes the stack pointer, clears flags and timing registers, configures input and output pins, sets up the PWM timers, configures the external interrupt for the parking sensor, and enables global interrupts. After initialization, the program enters the main loop and continuously performs sensor reading, marker detection, lost line checking, parking detection, and movement control.

The main program structure can be summarized as:

- reset and initialization
- sensor reading
- line-following logic
- parking detection logic
- parking subroutine
- stop or parked state with safety timeout

This modular structure helped with debugging and explanation during development.

3.2 Register Allocation

The main working registers used in the implementation are shown below.

Register	Purpose
r16 (temp)	Temporary working register
r17 (sensors)	Stores sensor readings from PINC
r18 (robotState)	Stores control flags such as bay detection, reduced-speed parking mode, and lost-line timer state
r24 (timerL)	Low byte of loop timer
r25 (timerH)	High byte of loop timer
r2 (lastMarkerL)	Low byte of previous marker time
r3 (lastMarkerH)	High byte of previous marker time
r8 (lostTimerL)	Low byte of lost-line timer start

r9 (lostTimerH)	High byte of lost-line timer start
r4 (leftBase)	Base speed value for the left motor
r5 (rightBase)	Base speed value for the right motor
r6 (turnFast)	Faster turning speed value
r7 (turnSlow)	Slower turning speed value

The robotState register stores the main control flags used by the system. Bit 0 is used as the bay detected flag, bit 1 is used as the reduced-speed or parking-stage flag, and bit 2 is used as the lost-line timer state. Additional timing registers are used to store the previous marker time and the start time of the lost-line condition. This makes it possible to detect markers, switch between normal and slow operating modes, and safely stop the robot if the line is lost for too long.

3.3 PWM-Based Motor Speed Control

Motor speed control was implemented using hardware PWM. Timer0 was configured for PWM on pin 5, and Timer1 was configured for PWM on pin 10. Both timers were used in Fast PWM mode with a prescaler of 64. Instead of keeping only fixed PWM values, the program stores the current forward and turning speeds in dedicated registers and updates them dynamically according to marker detection.

This allowed the robot to use:

- normal speed values during standard line following,
- reduced speed values while the parking-stage flag is active,
- different speed pairs for left and right turning,
- and separate values for the parking routine.

The use of hardware PWM helped the robot move more smoothly and allowed speed balancing between the two motors. Marker timing logic was used to switch between normal-speed and reduced-speed operation, while the lost-line timer provided a safety stop if the robot remained away from the expected tracking condition for too long.

4. System Behavior Model

The behavior of the robot can be represented as a simple state-based system. After power-on, the robot enters an initialization state. Once all registers, ports, timers, and interrupts are configured, the system moves into the line-following state. In this state, the robot continuously reads the left and right sensor inputs and adjusts the motors according to the detected sensor pattern.

While line following is active, the program also watches for marker detections and the external interrupt on D3. Marker timing is used to switch between normal-speed operation and reduced-speed parking-stage operation. When the parking-stage flag is active and the correct interrupt signal is detected, the system transitions into the parking routine. In addition, if the robot remains in the lost-line condition for too long while the parking-stage flag is inactive, the system enters a permanent stop state as a safety measure. In parking mode, the robot performs a fixed sequence of movements to align itself, rotate left, reverse into the bay, and then stop permanently.

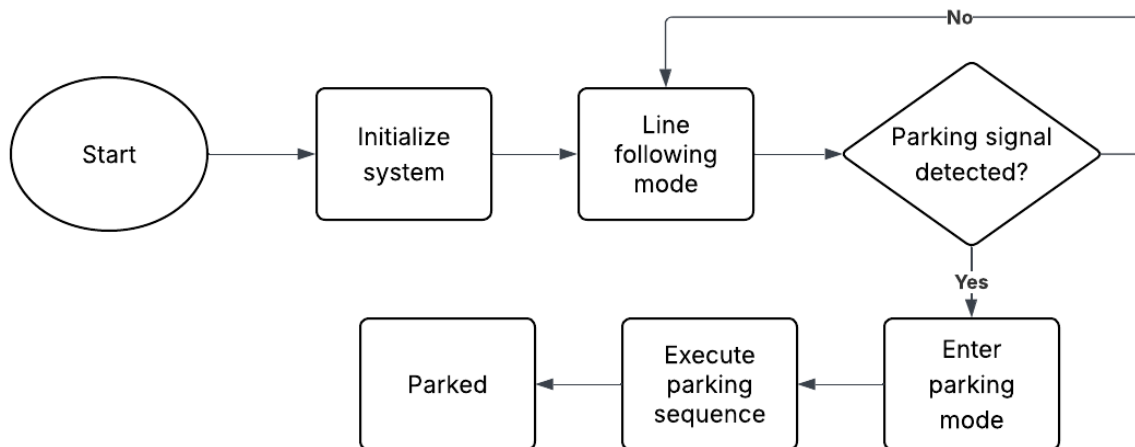


Figure 2: State-Based Behavior of the Robot

5. Algorithm Flow Summaries

5.1 Line Following Process

The program keeps reading the two line sensors from PINC. If one sensor shows that the robot has moved away from the line, the robot turns to correct its path. If a valid marker is detected, the program checks the marker timing, changes the operating mode if needed, and then keeps moving forward. If both sensors show that the line is lost, the program starts lost-line handling and stops the robot if that condition continues for too long. This logic is controlled using branch instructions inside the main loop.

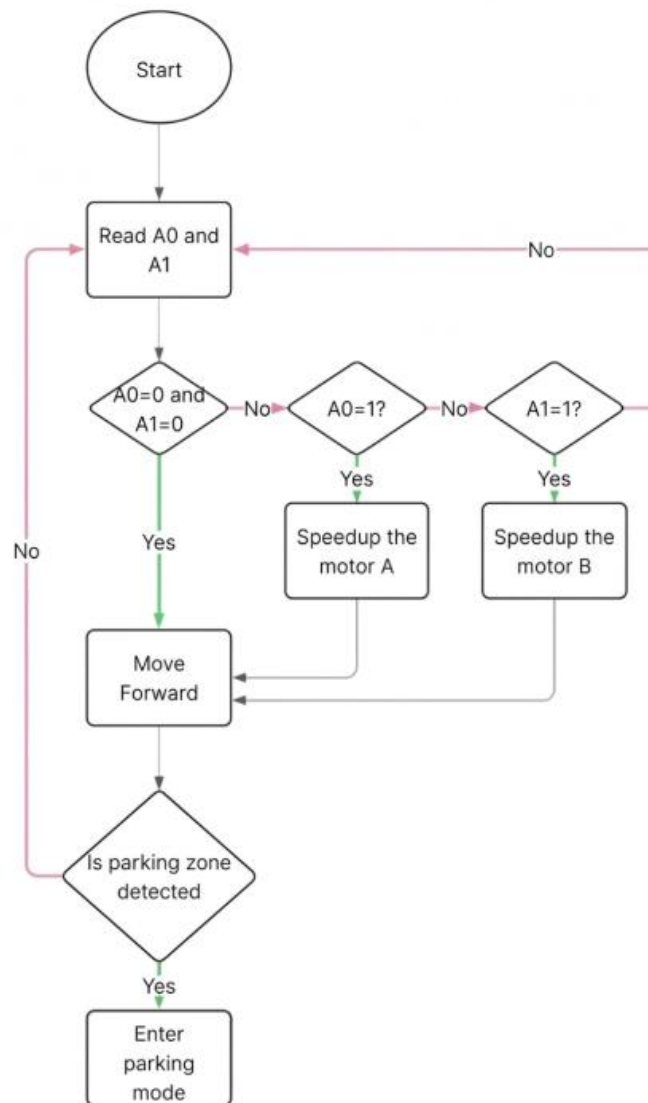


Figure 3: Line Following Flow Summary

5.2 Parking Detection Process

The parking detection uses the interrupt on pin D3. When the sensor is triggered, the program sets a bay-detected flag. The robot only accepts this signal when it is already in the parking stage. If the signal comes at the wrong time, it is ignored. If it comes during the parking stage, the robot starts the parking routine. This helps prevent wrong parking.

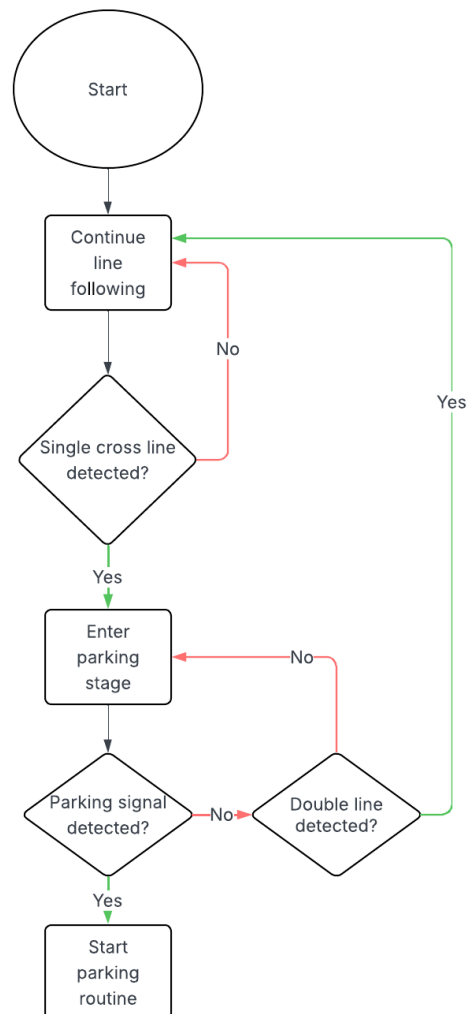


Figure 4: Parking Detection Flow Summary

5.3 Parking Execution Process

Once the valid parking condition is confirmed, the robot executes a fixed parking sequence. It first moves forward slightly, then stops, rotates left, stops again, reverses into the bay, and finally stops permanently. This sequence was implemented through a dedicated subroutine and calibrated using delay routines.

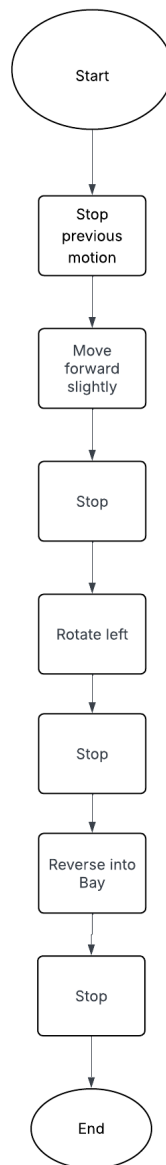


Figure 5: Flowchart of the Parking Execution Process

6. System Implementation

6.1 Line Following Implementation

The line-following logic was implemented using the two TCRT5000 sensors connected to A0 and A1. At the register level, these are read through PC0 and PC1. After masking the input values, the main loop checks the sensor pattern and decides how the robot should respond. One-side sensor patterns cause left or right correction, marker patterns trigger marker timing logic, and a lost-line pattern starts the lost-line timer.

In the forward routine, the left and right motors are set to rotate forward with slightly different PWM values to compensate for motor mismatch. In the turning routines, one motor is slowed and the other motor is speeded up. The program switches between normal-speed and reduced-speed values depending on the current operating mode.

6.2 Parking Detection Implementation

Parking detection was implemented using INT1 on PD3, which corresponds to Arduino D3. In the latest implementation, the external interrupt was configured to trigger on a rising edge. When triggered, the ISR sets the bay detected flag inside the robotState register. The CHECK_PARK routine then clears the flag and only starts the parking sequence if the reduced-speed parking-stage flag is active.

This means the robot does not park simply because an interrupt is received. It only parks when the interrupt is detected during the correct stage of operation. This makes the parking response more reliable and reduces the chance of starting the parking sequence at the wrong location.

6.3 Parking Maneuver Implementation

The parking routine was written as a dedicated subroutine called from the main loop. The routine performs the following movements:

- short forward alignment
- short stop
- left rotation
- short stop
- reverse into the bay
- final stop

These movements are created by changing the motor direction bits and adjusting the PWM values. Delay subroutines such as 200 ms and 500 ms were used to control the movement durations. The reversing part of the sequence is performed by repeatedly calling the 500 ms

delay routine to obtain a longer reverse period. After the robot parks, the motors are stopped and the system remains in a permanent stop state.

7. Testing, Results, and Evaluation

7.1 Test Cases and Results

Test No	Scenario	Expected Result	Actual Result	Status
1	Forward movement	Vehicle moves forward correctly	Stable forward movement observed	Pass
2	Straight line following	Robot remains on track	Correct tracking observed	Pass
3	Curved line following	Robot follows bends in track	Turning correction worked	Pass
4	Marker detection and mode switching	A valid marker changes the operating mode, and the next valid marker restores normal mode	Marker detection and mode switching observed	Pass
5	Line-loss timeout	Robot stops when the prolonged non-tracking condition exceeds about 3 seconds	Safety timeout stop observed	Pass
6	Parking interrupt detection	Selected bay signal is detected during parking stage	Correct interrupt response observed	Pass
7	Parking maneuver	Robot enters selected bay	Successful parking observed	Pass
8	Full system run	Complete line-following and parking sequence	Full operation completed	Pass

7.2 System Performance Review

The robot showed stable movement in straight sections and acceptable correction during curves. The use of PWM allowed smoother speed control, and the slightly different PWM values for the two motors helped compensate for motor imbalance. The interrupt-based parking detection improved responsiveness, and the timer-based marker logic allowed the system to switch reliably between normal-speed and reduced-speed operation.

The parking sequence worked fully in the completed system. Because the movement sequence is based on timed delays, the final result depends on calibration, battery level, and surface conditions. However, after tuning the delay values, speed values, and marker timing thresholds, the system was able to park inside the correct bay reliably during testing.

7.3 Issues Faced and Improvements

Several practical issues were observed during development and testing.

One issue was motor speed mismatch. Even when similar PWM values were applied, the left and right motors did not rotate at exactly the same speed. This caused slight drifting during forward motion. To reduce this effect, slightly different PWM values were assigned to the two motors.

Another issue was sensor sensitivity. The TCRT5000 sensors were affected by factors such as floor brightness, tape darkness, ambient lighting, and sensor height from the ground. This made calibration important, especially when the robot moved through curved sections of the track.

A third issue was marker and timeout calibration. The program uses the time gap between marker detections to switch operating modes, and it also uses a lost-line timeout to stop the robot safely if the condition persists for too long. Therefore, these timing thresholds had to be tuned carefully through repeated testing on the actual track.

The following improvements were made during implementation:

- forward motor PWM values were balanced to reduce drifting,
- sensor placement and calibration were adjusted for more stable detection,
- marker timing thresholds were tuned for reliable operating-mode switching,

lost-line timing behavior was tuned to stop the robot safely when tracking was not recovered in time.

Possible future improvements include:

using more precise feedback for parking,

replacing software delays with more timer-based calibration,

adding better filtering or threshold adjustment for sensors.

8. Physical Build Evidence

8.1 Top View

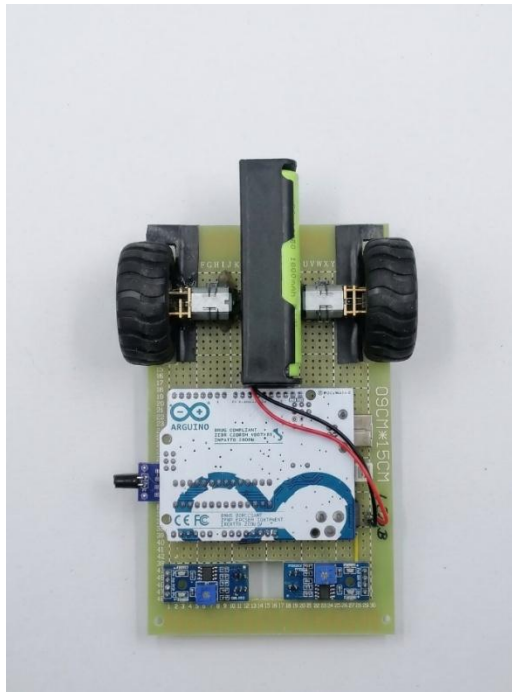


Figure 6: Top view of the robot

8.2 Wiring Layout

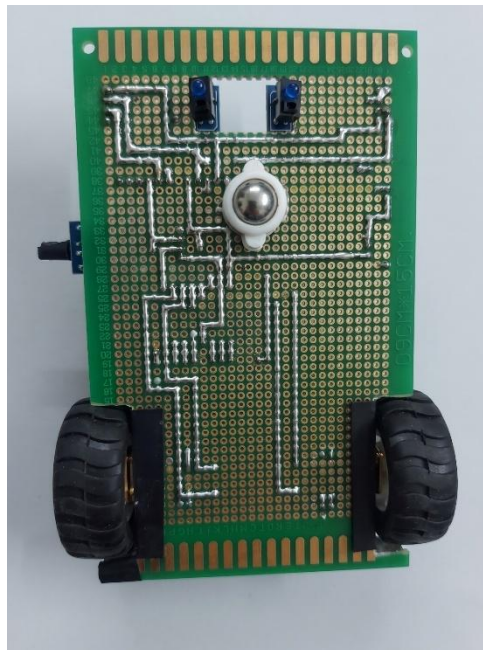


Figure 7: wiring layout of the robot

9. Discussion

This project demonstrated how a low-level embedded system can be built to perform autonomous movement and decision making. One of the main strengths of the project was that all control was implemented directly at register level in AVR Assembly language. This provided a strong understanding of the ATmega328P architecture, including timers, interrupts, data direction registers, port reading, and PWM generation.

Another important lesson from the project was the relationship between software and hardware. The robot's behavior depended not only on the correctness of the code, but also on physical factors such as motor characteristics, battery voltage, chassis balance, wheel traction, and sensor alignment. This meant that repeated testing and calibration were necessary before reliable results could be obtained.

The final design achieved the assignment goals using a practical approach. The system successfully combined line following, interrupt-based parking detection, marker-based mode switching, lost-line safety handling, and a timed parking sequence into one working autonomous car.

10. Conclusion

In conclusion, this project successfully achieved the main requirements of the IE3064 Embedded Systems Design assignment. The developed Arduino Uno based robot was able to follow the marked line track, react to sensor inputs, switch operating mode using marker detection, respond to the parking signal through the external interrupt on pin D3, and perform the final parking maneuver inside the correct bay. The system also included a safety timeout feature that stopped the robot if the lost-line condition persisted for too long outside the parking stage.

The project provided practical experience in AVR Assembly programming, direct register manipulation, PWM motor control, interrupt handling, sensor interfacing, timing-based decision making, and autonomous vehicle control.

11. References

- [1] Microchip Technology Inc., *ATmega328P: 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash, Datasheet*, 7810D–AVR–01/15, Jan. 2015.
- [2] Toshiba Electronic Devices & Storage Corporation, *TB6612FNG: Driver IC for Dual DC Motor, Data Sheet*, Oct. 2014.
- [3] Arduino, “UNO R3 | Arduino Documentation.”
- [4] Microchip Technology Inc., *AVR Instruction Set Manual*, DS40002198A.

12. Appendix

Individual Contribution

LKCM Perera – IT23619494

Was mainly responsible for the firmware side of the project. This included developing the AVR Assembly program, configuring the registers, timers, PWM control, and interrupt handling, and implementing the line-following logic, marker-based mode switching, lost-line timeout handling, and parking sequence. Member 1 also contributed to software testing, debugging, and tuning the speed and timing values to improve the overall behavior of the robot.

KMTP Kulasinha – IT23749566

was mainly responsible for the hardware side of the project. This included assembling the robot chassis, integrating the Arduino Uno, TB6612FNG motor driver, sensors, motors, and battery supply, and checking the wiring and pin connections. Member 2 also contributed to sensor positioning, physical calibration, hardware testing, and preparation of the schematic, photographs, and other documentation related to the final system.